

Scenario 1: A streaming pipeline accidentally inserted duplicate user click events into your raw database. Write a query to keep only the **most recent** click for each unique user

Solution: Use the ROW_NUMBER() window function to partition by the user and order by the timestamp descending, then filter out any row index greater than 1.

Scenario 2: You hit a REST API and get back a list of records. Some records have missing pricing data (None). Write a Python snippet to clean this data before loading it into a DataFrame.

Solution: Filter the list using a list comprehension or a loop, replacing missing values with a sensible default or dropping them entirely based on business logic.

Scenario 3: The interviewer asks: "Why is idempotency important in data pipelines, and how do you achieve it?"

Solution:

- **Definition:** An idempotent pipeline yields the exact same data warehouse state whether it runs once or ten times with the same input data.
 - **Why it matters:** If an Airflow task fails halfway through and retries, you must avoid creating duplicate rows.
 - **How to achieve it:** Use UPSERT (Merge) statements instead of blind INSERT INTO statements, or use atomic directory overwrites (e.g., INSERT OVERWRITE in Spark/Hive) for file-based storage.
-

Scenario 4: A daily batch pipeline has suddenly started failing because the source file size doubled overnight, causing out-of-memory (OOM) errors. How do you triage this?

Solution:

- **Immediate Fix:** Increase the memory allocation of your worker/container or change the compute cluster size to clear the current day's logjam.
- **Long-term Fix:** Switch the pipeline processing pattern from **loading the entire file into memory** (like standard pandas.read_csv()) to **chunking / streaming processing** (using chunksize in Pandas, or switching the task to Apache Spark/Dask which handles distributed, disk-spilled processing automatically).